

Which data structure is followed by dictionary in python



Python dictionary follows the Hash Table data structure internally.

How it works:

- Each key is passed through a hash function.
- The hash value determines where the key-value pair is stored in memory.
- This allows very fast operations.

Time Complexity

Operation	Average Time
Search by key	$O(1)$
Insert	$O(1)$
Delete	$O(1)$

Parameters for insertion creation deletion searching traversal

Agar tum Insertion, Deletion, Searching, Traversal ke basis par List aur Dictionary compare karna chahte ho, to ye table yaad rakh lo:

Operation	List	Dictionary
Insertion	$O(1)$ at end (<code>append()</code>), $O(n)$ at middle	$O(1)$ average
Deletion	$O(n)$	$O(1)$ average
Searching	$O(n)$	$O(1)$ average
Traversal	$O(n)$	$O(n)$

2. Deletion

List

```
</> Python  
lst.remove(30)
```



▶ Run

Pehle 30 ko dhoondho, phir elements shift karo.

→ $O(n)$

Dictionary

```
</> Python  
del d["name"]
```



▶ Run

Hash se direct key mil jaati hai.

→ $O(1)$ average

1. Insertion

List

```
</> Python  
lst = [10, 20, 30]  
lst.append(40)
```



▶ Run

End me add karna fast hota hai → $O(1)$

Lekin beech me:

```
</> Python  
lst.insert(1, 15)
```



▶ Run

Elements shift karne padte hain → $O(n)$

Dictionary

```
</> Python  
d = {}  
d["name"] = "Rahul"
```



▶ Run



Hash table me direct store hota hai → $O(1)$ average

3. Searching

List

 Python



 Run

```
50 in lst
```

Ek-ek element check hoga.

→ $O(n)$

Dictionary

 Python



 Run

```
"name" in d
```

Hash calculate karke direct lookup.

→ $O(1)$ average

4. Traversal

List

 Python



 Run

```
for x in lst:  
    print(x)
```

Saare elements visit karne padenge.

→ $O(n)$

Dictionary

 Python



 Run

```
for key, value in d.items():  
    print(key, value)
```

Saari entries visit karni padengi.

→ $O(n)$

Exam Point of View

Data Structure	Insertion	Deletion	Searching	Traversal
List	$O(1) / O(n)$	$O(n)$	$O(n)$	$O(n)$
Dictionary	$O(1)$	$O(1)$	$O(1)$	$O(n)$

Reason:

- **List** → Dynamic Array
- **Dictionary** → Hash Table

Isliye searching aur deletion ke cases me dictionary generally list se faster hoti hai.

Which algorithm is followed for insertion in dictionary before python 3.6 and after 3.6

Ye thoda deep internals ka question hai.

Before Python 3.6

Dictionary ek hash table use karti thi.

Structure kuch aisa tha:

Hash Table

A diagram of a 1D lattice with two sites labeled A and B. The lattice is represented by a horizontal line with vertical tick marks at regular intervals. The sites are labeled A and B, and the bonds between them are represented by dashed lines.

Insertion steps:

1. Key ka hash calculate karo.
2. Hash ke basis par index nikalo.
3. Wahan store karo.
4. Agar collision ho, to probing (open addressing) use karo.

Example:

`</> Python`



`▶ Run`

```
d["name"] = "Rahul"
```

Internally:

```
hash("name")
```



```
index
```



```
store
```



Result

- Average insertion: $O(1)$
- Lookup: $O(1)$
- Order maintain nahi hota tha (implementation detail)

Python 3.6

Python developers ne **Compact Dictionary** introduce ki.

Main idea:

Instead of storing everything directly in hash table, dictionary ko do parts me divide kiya gaya:

Sparse Hash Table (indices)

```
[2, -, 0, 1]
```



Dense Entries Array

```
0 -> ("name", Rahul)
```

```
1 -> ("age", 20)
```

```
2 -> ("city", Delhi)
```



Insertion hamesha dense array ke end me hota tha.

Isliye insertion order automatically preserve ho gaya.

Python 3.7+

Insertion order officially language guarantee ban gaya.

 Python

```
d = {}  
d["C"] = 1  
d["A"] = 2  
d["B"] = 3  
  
print(d)
```



 Run

Output:

 Python

```
{'C': 1, 'A': 2, 'B': 3}
```



 Run

Hamesha same order.

Kya Algorithm Change Hua?

Hashing algorithm nahi badla.

Dictionary aaj bhi:

- Hash Table use karti hai
- Open Addressing use karti hai
- Collision resolution use karti hai

Badla kya?

☒ Data storage layout

Before 3.6

Hash Table

└─ Key + Value directly stored



After 3.6

Hash Table

└─ Entry Index



Dense Entry Array

└─ Key + Value stored sequentially



After 3.6

Hash Table

└─ Entry Index



Dense Entry Array

└─ Key + Value stored sequentially

Isi compact-dict design ki wajah se insertion order preserve hota hai.

Short Interview Answer

Python 3.6 se pehle dictionaries traditional hash table implementation use karti thi jisme insertion order guaranteed nahi tha. Python 3.6 me Compact Dictionary implementation introduce hui, jisme hash table ke saath ek dense entries array maintain ki gayi. Hashing aur open addressing wahi rahe, lekin entries insertion order me store hone lagi. Python 3.7 se insertion order preservation language guarantee ban gaya.